

Thuật toán Parallel BFS Hybrid

Trình bày Lý thuyết Toán học Đầy đủ

Tài liệu này trình bày và giải thích chi tiết thuật toán BFS song song lai (Hybrid MPI + OpenMP) kết hợp kỹ thuật Direction-Optimizing từ đầu đến cuối, sử dụng ký hiệu toán học hàn lâm kết hợp diễn giải bằng lời.

1 Bài toán đặt ra

Bài toán BFS (Breadth-First Search): Cho một đồ thị và một đỉnh nguồn, tìm **đường đi ngắn nhất** (tính theo số cạnh) từ đỉnh nguồn đến mọi đỉnh khác trong đồ thị.

Thách thức: Khi đồ thị có hàng triệu hoặc hàng tỷ đỉnh, một máy tính đơn lẻ không đủ sức tính toán trong thời gian hợp lý. Do đó, cần phân tán bài toán lên **nhiều máy tính** (Cluster) để nhiều máy cùng tính toán song song, nhằm giảm thời gian xử lý.

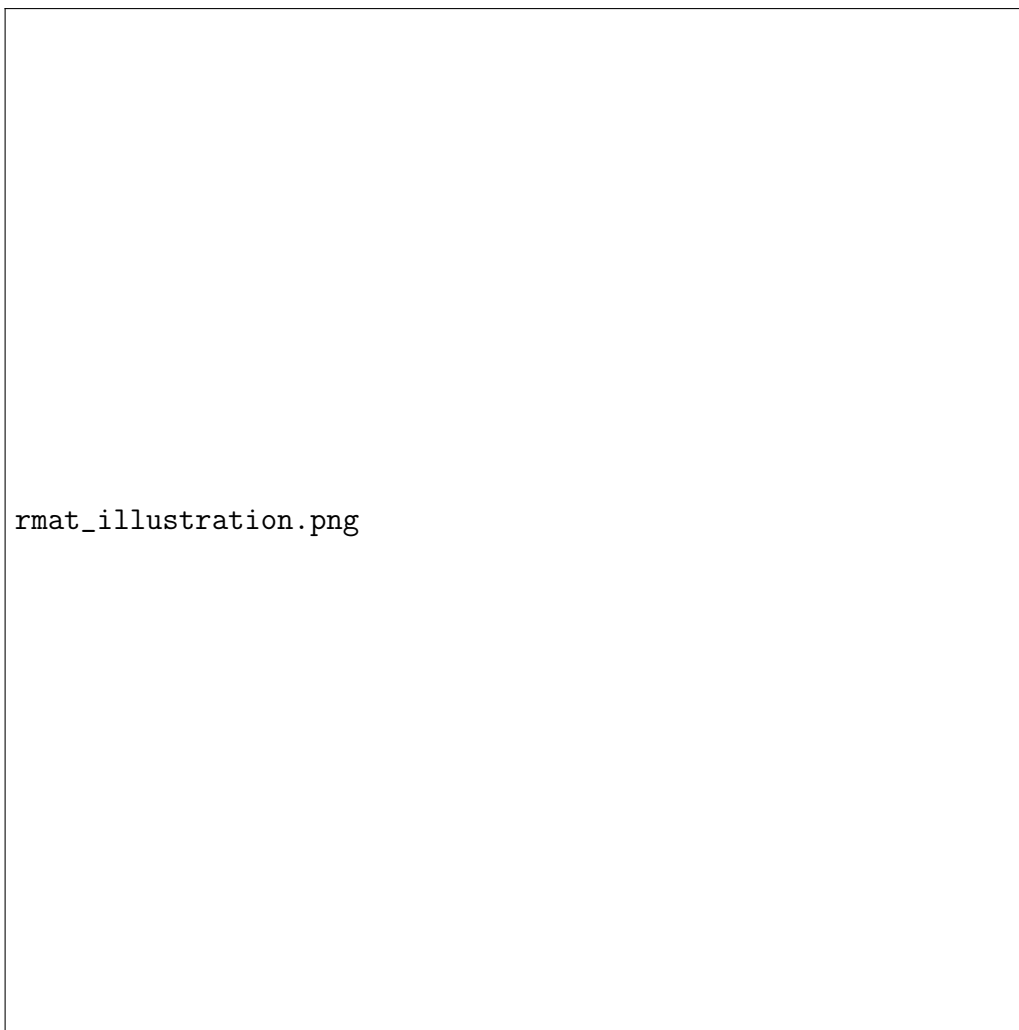
2 Thuật toán Sinh Đồ thị R-MAT

Trong dự án này, đồ thị đầu vào cho thuật toán BFS không được đọc từ file mà được **sinh tự động** bằng mô hình **R-MAT** (Recursive MATrix). Đây là mô hình tiêu chuẩn được sử dụng trong bảng xếp hạng siêu máy tính **Graph500**.

2.1 Ý tưởng cốt lõi

R-MAT sinh đồ thị bằng cách mô phỏng quá trình **đệ quy chia 4 góc phần tư** (recursive quadrant partitioning) trên ma trận kờ $n \times n$.

Hãy tưởng tượng ta có một ma trận kờ kích thước $n \times n$ (hàng đại diện cho đỉnh nguồn u , cột đại diện cho đỉnh đích v). Thay vì đặt cạnh ngẫu nhiên đều, R-MAT chia ma trận thành **4 góc phần tư** và gán xác suất khác nhau cho mỗi góc. Góc nào có xác suất cao hơn thì cạnh sẽ rơi vào đó nhiều hơn, tạo ra sự “tập trung” giống mạng xã hội thật.



Hình 1: Minh họa thuật toán R-MAT: ma trận kề được chia đệ quy thành 4 góc với xác suất (a, b, c, d) . Qua mỗi vòng lặp, thuật toán chọn một góc và thu nhỏ phạm vi cho đến khi xác định được một ô duy nhất — đó chính là cạnh (u, v) .

2.2 Tham số R-MAT

Mô hình R-MAT được xác định bởi 4 tham số xác suất (a, b, c, d) thỏa mãn:

$$a + b + c + d = 1.0, \quad a \geq b \geq c \geq d \tag{1}$$

Tham số	Góc phần tư	Giá trị Graph500	Ý nghĩa
a	Trên–Trái	0.57	Xác suất cạnh rơi vào vùng “hub nối hub” (đỉnh chỉ số nhỏ nối đỉnh chỉ số nhỏ). Đây là giá trị lớn nhất, tạo ra cụm đỉnh có bậc rất cao.
b	Trên–Phải	0.19	Xác suất cạnh nối hub (chỉ số nhỏ) sang vùng ngoại vi (chỉ số lớn).
c	Dưới–Trái	0.19	Xác suất cạnh nối ngoại vi sang hub (đối xứng với b).
d	Dưới–Phải	0.05	Xác suất cạnh nối ngoại vi với ngoại vi. Giá trị rất nhỏ, nghĩa là vùng đỉnh chỉ số lớn rất ít kết nối với nhau.

Tại sao a lớn nhất? Vì $a = 0.57$ (chiếm 57% xác suất), phần lớn cạnh sẽ rơi vào góc trên–trái. Điều này có nghĩa là các đỉnh có chỉ số nhỏ (0, 1, 2, ...) sẽ được kết nối dày đặc với nhau, tạo thành các “hub” — giống như những người nổi tiếng trên mạng xã hội có hàng triệu kết nối. Các đỉnh chỉ số lớn (ngoại vi) thì rất ít cạnh, giống như người bình thường chỉ có vài chục bạn bè.

2.3 Thuật toán Chi tiết: Sinh một Cạnh (u, v)

Đầu vào: Số đỉnh n (phải là lũy thừa của 2), bộ tham số (a, b, c, d) , bộ sinh số ngẫu nhiên.

Đầu ra: Một cạnh (u, v) với $u, v \in \{0, 1, \dots, n-1\}$.

Quy trình:

1. **Khởi tạo:** Đặt $u \leftarrow 0, v \leftarrow 0, \text{step} \leftarrow n$.

Ban đầu, “con trỏ” đang ở góc trên–trái của ma trận kề, và phạm vi xét là toàn bộ ma trận $n \times n$.

2. **Lặp** (trong khi $\text{step} > 1$):

- a. Chia đôi phạm vi: $\text{step} \leftarrow \text{step}/2$.
- b. Sinh một số ngẫu nhiên $r \in [0, 1)$.
- c. Chọn góc phần tư dựa vào r :
 - Nếu $r < a$: chọn góc **Trên–Trái** $\rightarrow u$ và v **không đổi**.
 - Nếu $a \leq r < a + b$: chọn góc **Trên–Phải** $\rightarrow v \leftarrow v + \text{step}$.
 - Nếu $a + b \leq r < a + b + c$: chọn góc **Dưới–Trái** $\rightarrow u \leftarrow u + \text{step}$.
 - Nếu $r \geq a + b + c$: chọn góc **Dưới–Phải** $\rightarrow u \leftarrow u + \text{step}, v \leftarrow v + \text{step}$.

Mỗi vòng lặp thu hẹp phạm vi ma trận xuống còn $1/4$. Nếu chọn góc Trên–Trái, ta ở nguyên ô hiện tại. Nếu chọn góc khác, ta dịch chuyển “con trỏ” (u, v) sang phần tương ứng. Sau $\log_2(n)$ vòng lặp, $\text{step} = 1$ và (u, v) xác định duy nhất **một ô** trong ma trận — đó chính là cạnh được sinh ra.

3. **Trả về** cạnh (u, v) .

2.4 Ví dụ Minh họa Cụ thể

Giả sử $n = 8$ (ma trận 8×8). Thuật toán chạy $\log_2(8) = 3$ vòng lặp:

Vòng	step	r ngẫu nhiên	Góc chọn	Kết quả
1	$8 \rightarrow 4$	$r = 0.35 (< a = 0.57)$	Trên–Trái	$u = 0, v = 0$. Phạm vi thu hẹp: ô $[0:3] \times [0:3]$.
2	$4 \rightarrow 2$	$r = 0.62 (a \leq r < a+b+c)$	Dưới–Trái	$u = 0 + 2 = 2, v = 0$. Phạm vi: $[2:3] \times [0:1]$.
3	$2 \rightarrow 1$	$r = 0.70 (\geq a+b)$	Dưới–Trái	$u = 2 + 1 = 3, v = 0$. Ô duy nhất: $(3, 0)$.

$\Rightarrow$ Cạnh được sinh ra là $(u, v) = (3, 0)$, nghĩa là đỉnh 3 nối với đỉnh 0.

2.5 Xây dựng Toàn bộ Đồ thị

Để sinh đồ thị $G = (V, E)$ hoàn chỉnh, thuật toán lặp lại quá trình trên $m_{\text{target}} = n \times \text{scale_factor}$ lần (trong đó $\text{scale_factor} = 16$ theo cấu hình thực nghiệm). Mỗi lần sinh được một cạnh (u, v) .

Sau khi sinh xong, thuật toán thực hiện các bước hậu xử lý:

1. **Loại bỏ self-loop:** Nếu $u = v$ thì bỏ qua cạnh đó (một đỉnh nối với chính nó là vô nghĩa trong BFS).
2. **Thêm cạnh ngược (Undirected):** Với mỗi cạnh (u, v) , thêm cả cạnh (v, u) vì đồ thị là vô hướng.
3. **Sắp xếp và loại bỏ cạnh trùng lặp (Deduplication):** Sắp xếp toàn bộ danh sách cạnh theo cặp (u, v) rồi loại bỏ các cạnh lặp. Đảm bảo mỗi cặp (u, v) chỉ xuất hiện đúng 1 lần.
4. **Chuyển đổi sang CSR:** Xây dựng mảng `row_ptr` và `adj` từ danh sách cạnh đã sạch.

2.6 Tính chất của Đồ thị R-MAT

Đồ thị R-MAT có các tính chất quan trọng ảnh hưởng trực tiếp đến hiệu năng song song:

- **Phân bố bậc lệch (Power-law degree distribution):** Một số ít đỉnh (hub) có bậc cực lớn (hàng nghìn đến hàng triệu cạnh), trong khi đại đa số đỉnh có bậc rất thấp (chỉ vài cạnh). Tính chất này mô phỏng chính xác mạng xã hội thực.
- **Thế giới nhỏ (Small-world property):** Đường kính (diameter) đồ thị rất nhỏ so với n , thuật toán BFS chỉ cần vài level (thường 5–7 level) là duyệt hết.
- **Tái tạo được (Reproducible):** Nhờ sử dụng **seed** cố định ($= 42$) cho bộ sinh số ngẫu nhiên (`xorshift64`), mọi lần chạy đều sinh ra đúng cùng một đồ thị. Điều này đảm bảo tính tái lập cho thực nghiệm khoa học.
- **Gây khó khăn cho cân bằng tải:** Do bậc lệch, khi chia đỉnh đều cho k tiến trình bằng 1D Partitioning, tiến trình quản lý vùng đỉnh chỉ số nhỏ (chứa hub) sẽ phải xử lý nhiều cạnh hơn hẳn tiến trình quản lý vùng đỉnh chỉ số lớn.

3 Các Định nghĩa và Ký hiệu Toán học

Trước khi đi vào thuật toán, ta cần thiết lập rõ ràng các khái niệm và ký hiệu sẽ sử dụng xuyên suốt tài liệu.

3.1 Đồ thị

Ký hiệu	Ý nghĩa	Giải thích
$G = (V, E)$	Đồ thị vô hướng	V là tập đỉnh, E là tập cạnh
$n = V $	Số đỉnh	Tổng số đỉnh trong đồ thị
$m = E $	Số cạnh	Tổng số cạnh (mỗi cạnh tính 2 chiều, nên thực tế lưu $2m$ entry)
$s \in V$	Đỉnh nguồn	Đỉnh xuất phát của thuật toán BFS

3.2 Quan hệ Lân cận

$$N(u) = \{v \in V \mid (u, v) \in E\} \quad (2)$$

Đây là **tập các đỉnh kề** (neighborhood) của đỉnh u , tức là tập hợp tất cả các đỉnh v sao cho tồn tại cạnh nối trực tiếp giữa u và v .

Bậc (degree) của đỉnh u là số lượng đỉnh kề của nó:

$$\deg(u) = |N(u)| \quad (3)$$

Ý nghĩa thực tế: Trong mạng xã hội, $N(u)$ là danh sách bạn bè của người u , và $\deg(u)$ là số bạn bè mà u có.

3.3 Hàm Khoảng cách

$$d : V \rightarrow \mathbb{N}_0 \cup \{-1\} \quad (4)$$

Hàm d gán cho mỗi đỉnh v một giá trị nguyên:

- $d(v) = -1$: đỉnh v **chưa được thăm** (chưa tìm thấy đường đi từ s đến v).
- $d(v) = k \geq 0$: đường đi ngắn nhất từ s đến v có độ dài đúng bằng k cạnh.

Giải thích: Ban đầu, tất cả đỉnh đều có $d(v) = -1$ (trạng thái “chưa biết”). Khi thuật toán khám phá được đỉnh nào, nó sẽ gán giá trị khoảng cách cụ thể cho đỉnh đó. Giá trị -1 được chọn vì nó luôn nhỏ hơn mọi khoảng cách hợp lệ (≥ 0), rất thuận tiện khi dùng phép max để gộp kết quả giữa các máy.

3.4 Tập Biên (Frontier)

$$F_l \subseteq V \quad (5)$$

F_l là tập hợp tất cả các đỉnh nằm ở **cấp độ** l (level l) — tức là các đỉnh có khoảng cách từ đỉnh nguồn s đúng bằng l :

$$F_l = \{v \in V \mid d(v) = l\} \quad (6)$$

Hình dung: Tưởng tượng ném một hòn đá xuống hồ nước. Gợn sóng hình tròn lan ra dần dần. F_l chính là **vòng tròn gợn sóng thứ l** — là ranh giới ngoài cùng của vùng đã được khám phá tại thời điểm đó. Thuật toán BFS hoạt động bằng cách liên tục đẩy vòng tròn này ra xa hơn, cho đến khi nó không thể lan rộng thêm nữa.

3.5 Hệ thống Phân tán

Ký hiệu	Ý nghĩa	Giải thích
k	Số tiến trình MPI	Tổng số tiến trình chạy trên Cluster
$P = \{p_0, p_1, \dots, p_{k-1}\}$	Tập các tiến trình	Mỗi p_i là một tiến trình MPI chạy trên một trong 3 máy
V_i	Phần đỉnh của p_i	Tập hợp con các đỉnh mà tiến trình p_i chịu trách nhiệm tính toán

3.6 Biểu diễn Dữ liệu: CSR (Compressed Sparse Row)

Đồ thị G được lưu trữ trong bộ nhớ bằng cấu trúc **CSR** gồm 2 mảng:

- **row_ptr** (kích thước $n + 1$): Mảng con trỏ hàng. Các đỉnh kề của đỉnh u được lưu liên tiếp trong mảng **adj** tại các vị trí từ **row_ptr**[u] đến **row_ptr**[$u+1$] - 1.
- **adj** (kích thước m): Mảng danh sách kề. Chứa tất cả các đỉnh kề, nối tiếp nhau.

Hay biểu diễn bằng toán học:

$$N(u) = \{\text{adj}[j] \mid j \in [\text{row_ptr}[u], \text{row_ptr}[u+1])\} \quad (7)$$

Tại sao dùng CSR? Đồ thị R-MAT rất thưa (sparse) — số cạnh $m \ll n^2$. Ma trận kề sẽ tốn $O(n^2)$ bộ nhớ với phần lớn là số 0. CSR chỉ tốn $O(n + m)$ và cho phép truy xuất danh sách lân cận trong $O(1)$ bước, tối ưu hóa hiệu suất cache.

3.7 Biểu diễn Tập Biên: Bitmap

Tập F_l được biểu diễn bằng một **mảng bitmap** B có $\lceil n/64 \rceil$ từ máy (word) 64-bit:

$$v \in F_l \iff B[\lfloor v/64 \rfloor] \& (1 \ll (v \bmod 64)) \neq 0 \quad (8)$$

Giải thích đơn giản: Mỗi đỉnh v được gán 1 bit trong mảng. Bit = 1 nghĩa là đỉnh đó đang nằm trong Frontier, bit = 0 nghĩa là không. Lý do dùng bitmap: (1) tra cứu $O(1)$, (2) hợp nhất giữa các máy cực kỳ nhanh bằng phép OR bitwise.

4 Phân rã Dữ liệu (Data Decomposition)

4.1 Sao chép Đồ thị (Graph Replication)

Toàn bộ cấu trúc CSR của đồ thị G (gồm **row_ptr** và **adj**) được **sao chép y hệt** vào bộ nhớ RAM của **tất cả** k tiến trình trên cả 3 máy.

Tại sao? Trong quá trình tính toán, tiến trình p_i cần truy xuất danh sách lân cận $N(v)$ của các đỉnh v bất kỳ (kể cả những đỉnh không thuộc V_i). Nếu đồ thị bị chia nhỏ, tiến trình sẽ phải gửi yêu cầu MPI_Send/Recv sang máy khác để hỏi “đỉnh v có những đỉnh kề nào?” — rất chậm. Nhờ sao chép, mọi tiến trình đều có bản đồ thị đầy đủ trong RAM cục bộ, tra cứu tức thì mà không cần giao tiếp mạng.

4.2 Phân hoạch Tập đỉnh (1D Block Partitioning)

Tập đỉnh $V = \{0, 1, \dots, n-1\}$ được **phân hoạch** thành k phần **rời nhau** (không giao nhau), mỗi phần giao cho một tiến trình:

$$V_i = \left[\left\lfloor \frac{n \cdot i}{k} \right\rfloor, \left\lfloor \frac{n \cdot (i+1)}{k} \right\rfloor \right) \quad (9)$$

Tính chất phân hoạch đảm bảo:

- **Bao phủ toàn bộ:** $\bigcup_{i=0}^{k-1} V_i = V$ (mọi đỉnh đều có người chịu trách nhiệm).
- **Không trùng lặp:** $V_i \cap V_j = \emptyset, \forall i \neq j$ (không có đỉnh nào bị hai tiến trình cùng tính).

Giải thích: Giả sử $n = 800,000$ đỉnh, $k = 8$ tiến trình. Thì: p_0 quản lý đỉnh $[0, 100,000)$; p_1 quản lý $[100,000, 200,000)$, v.v. Mỗi tiến trình chỉ **tính toán và cập nhật** giá trị $d(v)$ cho các đỉnh $v \in V_i$ thuộc phần của mình (Owner-Computes Rule).

5 Thuật toán Chi tiết Từng Bước

5.1 Bước 1: Khởi tạo (Initialization)

Mỗi tiến trình $p_i \in P$ thực hiện **đồng thời** và **độc lập** các bước sau:

1. **Khởi tạo khoảng cách:**

$$d(v) \leftarrow -1, \quad \forall v \in V \quad (10)$$

Đánh dấu tất cả đỉnh là “chưa thăm”.

2. **Gán đỉnh nguồn:**

$$d(s) \leftarrow 0 \quad (11)$$

Đỉnh nguồn s có khoảng cách đến chính nó bằng 0.

3. **Khởi tạo Frontier:**

$$F_0 \leftarrow \{s\} \quad (12)$$

Ban đầu, vòng sóng chỉ chứa mỗi mình đỉnh nguồn.

4. **Khởi tạo biến đếm:**

$$l \leftarrow 1, \quad m_{unvisited} \leftarrow m \quad (13)$$

l là cấp độ hiện tại (sẽ gán cho các đỉnh mới khám phá). $m_{unvisited}$ là tổng số cạnh chưa bị duyệt qua, ban đầu bằng toàn bộ số cạnh.

5.2 Bước 2: Vòng lặp Duyệt theo Cấp độ (Level-synchronous Loop)

Vòng lặp chính của thuật toán. Tại mỗi cấp độ l , thuật toán đẩy “vòng sóng” F_l ra xa thêm một bước, sinh ra vòng sóng mới F_{l+1} .

Điều kiện dừng: Vòng lặp tiếp tục chừng nào $F_l \neq \emptyset$ (còn đỉnh để khám phá).

5.2.1 Bước 2.1: Đánh giá Heuristic — Quyết định Hướng duyệt (Direction-Optimizing)

Đây là bước **thông minh nhất** của thuật toán, dựa trên nghiên cứu của Beamer et al. (2012). Mục đích: chọn chiến lược duyệt tối ưu nhất cho cấp độ hiện tại.

Ý tưởng cốt lõi: Không phải lúc nào cũng nên duyệt theo cùng một cách. Tùy theo kích thước Frontier, có lúc duyệt xuôi (Top-Down) hiệu quả hơn, có lúc duyệt ngược (Bottom-Up) hiệu quả hơn.

Thực hiện tính toán:

Mỗi tiến trình p_i đếm **cục bộ** tổng số cạnh đang nối ra từ các đỉnh Frontier mà nó có trong phần dữ liệu:

$$c_i = \sum_{u \in F_i} \deg(u) \quad (14)$$

Mỗi tiến trình tính tổng bậc của tất cả đỉnh đang nằm trong Frontier. Đây là số cạnh mà thuật toán sẽ phải kiểm tra nếu duyệt theo kiểu Top-Down.

Sau đó, gộp tổng từ tất cả các tiến trình bằng phép **Reduction toàn cục** (Global Sum):

$$C = \sum_{i=0}^{k-1} c_i \quad (\text{thông qua MPI_Allreduce với MPI_SUM}) \quad (15)$$

Hàm `MPI_Allreduce(MPI_SUM)` sẽ cộng các giá trị c_i của tất cả tiến trình lại, và kết quả C được phân phối ngược lại cho mọi tiến trình.

Quy tắc quyết định:

Thuật toán duy trì một biến trạng thái xác định đang duyệt theo hướng nào. Hai hằng số $\alpha = 14$ và $\beta = 24$ được chọn theo nghiên cứu gốc:

- Nếu đang ở chế độ **Top-Down**, kiểm tra:

$$C > \frac{m_{unvisited}}{\alpha} \quad (16)$$

- Nếu **đúng** $\rightarrow$ Frontier đang bùng nổ quá lớn, sẽ phải kiểm tra rất nhiều cạnh một cách lãng phí $\rightarrow$ Chuyển sang **Bottom-Up**.
- Nếu **sai** $\rightarrow$ Giữ nguyên **Top-Down**.

- Nếu đang ở chế độ **Bottom-Up**, kiểm tra:

$$|F_l| \geq \frac{n}{\beta} \quad (17)$$

- Nếu **đúng** $\rightarrow$ Frontier vẫn còn lớn $\rightarrow$ Giữ nguyên **Bottom-Up**.
- Nếu **sai** $\rightarrow$ Frontier đã thu nhỏ đủ rồi $\rightarrow$ Quay lại **Top-Down**.

Giải thích trực giác: Hãy tưởng tượng bạn đang tìm kiếm một người trong thành phố. Nếu bạn chỉ biết vài người (Frontier nhỏ), bạn sẽ hỏi từng người đó “bạn quen ai?” (Top-Down). Nhưng nếu bạn đã biết hầu hết cư dân (Frontier khổng lồ), thì thay vì hỏi hàng triệu người đã biết, tốt hơn là đi đến từng người lạ còn lại và hỏi “bạn có quen ai trong danh sách không?” (Bottom-Up) — vì số người lạ bây giờ rất ít.

5.2.2 Bước 2.2: Tính toán Cục bộ Song song (Local Parallel Computation)

Đây là pha mà mỗi tiến trình **tính toán độc lập** (không giao tiếp mạng). Mục tiêu: sinh ra tập biên tiếp theo F_{l+1} .

Khởi tạo: $F_{l+1}^{(i)} \leftarrow \emptyset$ (tập biên mới của tiến trình i , ban đầu rỗng).

Trường hợp A: Top-Down (Duyệt xuôi) **Ý tưởng:** “Đi từ Frontier ra ngoài” — Lấy từng đỉnh đã biết trong Frontier, xem nó có hàng xóm nào chưa được thăm không.

Mô tả toán học: Chia tập F_l thành các phần bằng nhau cho k tiến trình. Tiến trình p_i xử lý phần frontier từ vị trí $\lfloor |F_l| \cdot i/k \rfloor$ đến $\lfloor |F_l| \cdot (i+1)/k \rfloor$.

Bên trong tiến trình p_i , các luồng OpenMP **chia nhau** duyệt song song các đỉnh frontier được giao:

$$\begin{aligned}
 &\forall u \in (\text{phần frontier của } p_i) : \\
 &\quad \forall v \in N(u) : \\
 &\quad \quad \text{Nếu } d(v) = -1 : \\
 &\quad \quad \quad d(v) \leftarrow l \\
 &\quad \quad \quad F_{l+1}^{(i)} \leftarrow F_{l+1}^{(i)} \cup \{v\}
 \end{aligned} \tag{18}$$

Diễn giải bằng lời: Với mỗi đỉnh u trong Frontier (những đỉnh vừa được tìm thấy ở bước trước), thuật toán lần lượt nhìn vào từng đỉnh kề v của u . Nếu v chưa từng được thăm (giá trị $d(v)$ vẫn bằng -1), thì:

- Gán khoảng cách cho v : $d(v) = l$ (đỉnh v cách đỉnh nguồn đúng l bước).
- Đưa v vào tập Frontier mới để xử lý tiếp ở bước sau.

Xử lý xung đột giữa các luồng (Race Condition): Nhiều luồng OpenMP có thể đồng thời phát hiện cùng một đỉnh v chưa thăm (từ các đỉnh u khác nhau). Để tránh ghi đè lẫn nhau, code sử dụng **Compare-And-Swap (CAS) nguyên tử**: `__sync_val_compare_and_swap(&dist[v], -1, level)`. Phép toán này đảm bảo chỉ DUY NHẤT MỘT luồng thành công ghi $d(v)$, các luồng còn lại sẽ phát hiện $d(v)$ đã bị thay đổi và bỏ qua.

Trường hợp B: Bottom-Up (Duyệt ngược) Ý tưởng: “Đi từ ngoài vào Frontier”
 — Thay vì duyệt từ Frontier ra, đi ngược lại: quét tất cả các đỉnh chưa thăm, xem nó có hàng xóm nào đang nằm trong Frontier không.

Mô tả toán học: Tiến trình p_i quét **toàn bộ dải đỉnh** mà nó sở hữu $V_i = [\text{local_start}, \text{local_end})$:

$$\begin{aligned}
 &\forall u \in V_i \text{ thỏa mãn } d(u) = -1 : \\
 &\quad \text{Nếu } \exists v \in N(u) \text{ sao cho } v \in F_l : \\
 &\quad \quad d(u) \leftarrow l \\
 &\quad \quad F_{l+1}^{(i)} \leftarrow F_l^{(i)} \cup \{u\} \\
 &\quad \text{break (dừng xét các } v \text{ còn lại — Symmetry Breaking)} \tag{19}
 \end{aligned}$$

Diễn giải bằng lời: Thuật toán quét từng đỉnh u chưa thăm trong phần dữ liệu của mình. Với mỗi đỉnh u như vậy, nó nhìn ngược vào danh sách hàng xóm $N(u)$ để tìm xem có ai trong đó đang nằm sẵn trong Frontier không. Nếu tìm thấy **dù chỉ một** hàng xóm v nằm trong Frontier, lập tức:

- Gán $d(u) = l$.
- Đưa u vào Frontier mới.
- **Dừng ngay (break)**, không cần kiểm tra thêm các hàng xóm còn lại.

Tại sao “break” lại quan trọng (Symmetry Breaking)? Vì ta chỉ cần biết “đỉnh u có đường đến Frontier không?” — câu trả lời là Có hoặc Không. Ngay khi tìm thấy câu trả lời Có (một hàng xóm trong Frontier), việc kiểm tra thêm là hoàn toàn dư thừa. Kỹ thuật này **cực kỳ hiệu quả** vì khi Frontier rất lớn (chiếm phần lớn đồ thị), xác suất tìm thấy một hàng xóm trong Frontier ngay ở vài bước đầu tiên là rất cao, tiết kiệm hàng triệu phép so sánh thừa.

Tại sao Bottom-Up hiệu quả khi Frontier lớn? Khi Frontier chứa hàng triệu đỉnh, phương pháp Top-Down phải duyệt hàng triệu đỉnh $\times$ hàng nghìn hàng xóm = hàng tỷ cạnh. Trong khi đó, Bottom-Up chỉ quét các đỉnh **chưa thăm** (số lượng ít khi Frontier lớn) và dừng ngay khi tìm thấy 1 liên kết. Tổng số phép tính giảm đi hàng chục lần.

5.2.3 Bước 2.3: Đồng bộ hóa Toàn cục (Global Synchronization)

Sau pha tính toán cục bộ, mỗi tiến trình p_i chỉ có:

- Mảng $d^{(i)}(v)$ với **một phần** đã được cập nhật (phần đỉnh của p_i), phần còn lại vẫn giữ giá trị cũ -1 .
- Tập biên cục bộ $F_{l+1}^{(i)}$ chỉ chứa các đỉnh do p_i phát hiện.

Bây giờ cần **hợp nhất** kết quả từ tất cả k tiến trình lại.

Phép đồng bộ 1: Hợp nhất Mảng khoảng cách

$$d(v) \leftarrow \max_{j=0}^{k-1} d^{(j)}(v), \quad \forall v \in V \quad (20)$$

Thực hiện qua `MPI_Allreduce(MPI_IN_PLACE, dist, n, MPI_INT, MPI_MAX, MPI_COMM_WORLD)`.

Giải thích tại sao dùng MAX: Với mỗi đỉnh v , chỉ có duy nhất 1 tiến trình (tiến trình sở hữu v) đã gán $d(v) = l > 0$. Tất cả các tiến trình khác vẫn giữ $d(v) = -1$. Phép $\max(-1, -1, \dots, l, \dots, -1) = l$. Vậy kết quả đúng sẽ được truyền đến tất cả các máy.

Hậu quả quan trọng: Sau bước này, **mọi** tiến trình trên cả 3 máy đều có **cùng một mảng** $d(v)$ **hoàn chỉnh và giống hệt nhau**. Đây là bất biến (invariant) cốt lõi của thuật toán.

Phép đồng bộ 2: Hợp nhất Tập biên

$$F_{l+1} \leftarrow \bigcup_{i=0}^{k-1} F_{l+1}^{(i)} \quad (21)$$

Thực hiện qua `MPI_Allreduce(MPI_IN_PLACE, bitmap, nwords, MPI_UINT64_T, MPI_OR, MPI_COMM_WORLD)`.

Giải thích: Mỗi tiến trình có một mảng bitmap riêng, trong đó bit $v = 1$ nếu p_i tìm thấy đỉnh v ở bước vừa rồi. Phép **OR** bitwise trên tất cả các bitmap sẽ gộp tất cả các đỉnh mới tìm được lại thành một Frontier duy nhất. Nếu p_2 tìm thấy đỉnh 5 và p_7 tìm thấy đỉnh 42, thì sau OR, cả bit 5 lẫn bit 42 đều bằng 1 ở mọi tiến trình.

Tính chất Rào cản (Barrier): Cả hai lệnh `MPI_Allreduce` đều là **Blocking** (chặn): tiến trình nào tính xong trước bắt buộc phải **đứng chờ** cho đến khi tiến trình chậm nhất hoàn thành. Sau đó, tất cả mới đồng loạt nhận kết quả đã hợp nhất và tiến sang bước tiếp theo. Đây chính là cơ chế **Global Barrier** đảm bảo tính đồng bộ tuyệt đối.

Chi phí: Mỗi lần gọi, hệ thống phải truyền mảng kích thước $O(n)$ qua mạng LAN giữa 3 máy. Đây là nguyên nhân chính gây thất cổ chai hiệu năng (bottleneck), vì tốc độ mạng LAN chậm hơn nhiều lần so với tốc độ CPU.

5.2.4 Bước 2.4: Cập nhật và Chuyển cấp độ

$$m_{unvisited} \leftarrow m_{unvisited} - \sum_{u \in F_{l+1}} \deg(u) \quad (22)$$

Trừ đi số cạnh gắn với các đỉnh vừa mới được thăm, cập nhật lại lượng cạnh chưa xử lý.

$$F_l \leftarrow F_{l+1} \quad (23)$$

“Vòng sóng mới” trở thành “vòng sóng hiện tại” cho bước tiếp theo.

$$l \leftarrow l + 1 \tag{24}$$

Tăng cấp độ lên 1.

Sau đó, quay lại **Bước 2.1** để bắt đầu cấp độ mới.

5.3 Bước 3: Kết thúc (Termination)

Thuật toán dừng tại cấp độ l_{max} khi:

$$F_{l_{max}} = \emptyset \quad (25)$$

Tức là: vòng sóng ngoài cùng không chứa đỉnh nào nữa — mọi đỉnh có thể đến được từ s đều đã được thăm hết.

Đầu ra: Mảng $d(v)$ hoàn chỉnh trên tất cả tiến trình, chứa khoảng cách đường đi ngắn nhất từ s đến mọi $v \in V$:

- $d(v) \geq 0$: Đỉnh v đến được từ s , khoảng cách đúng bằng $d(v)$.
- $d(v) = -1$: Đỉnh v không thể đến được từ s (nằm ở thành phần liên thông khác).

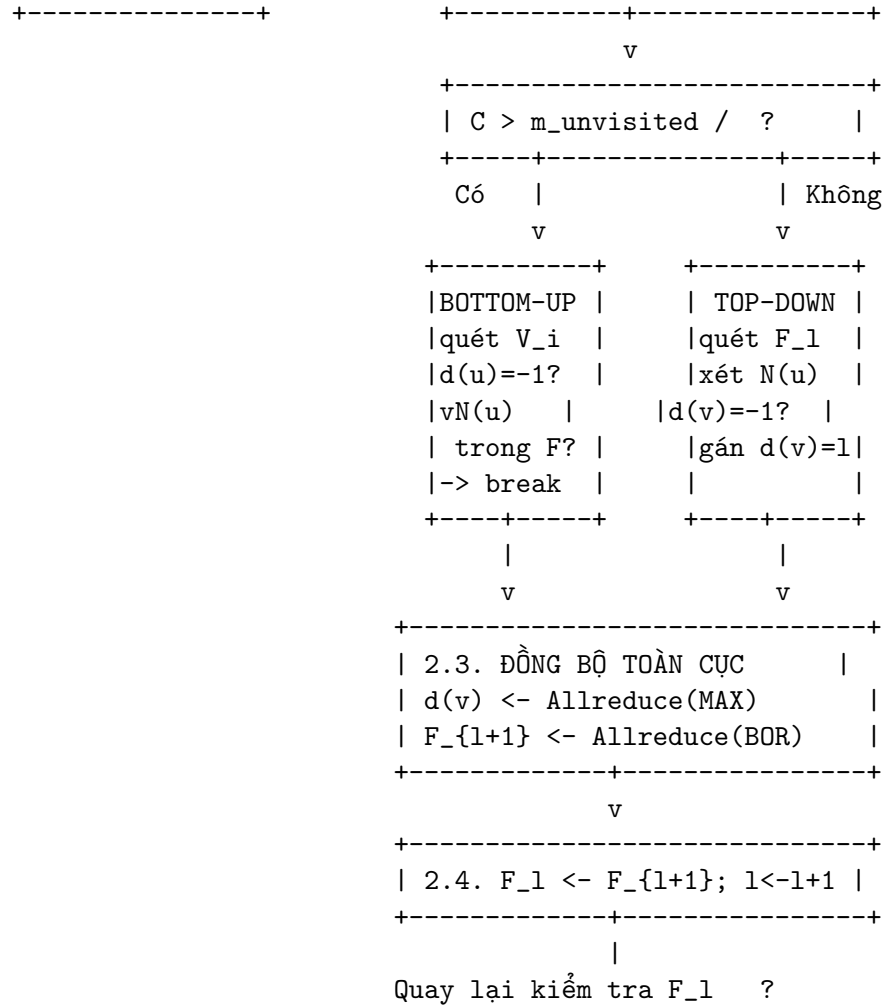
Tiến trình p_0 (Rank 0) chịu trách nhiệm xuất kết quả cuối cùng (in ra màn hình hoặc lưu file CSV).

6 Tổng hợp: Dòng chảy Toàn bộ Thuật toán

```

+-----+
|  KHỎI TẠO: d(v)<--1  v;  d(s)<-0;  F<-{s};  l<-1      |
+-----+
|
|      v
|
+-----+      Có
|  F_1  ?  |-----+
+-----+      |
|      |      v
|      |      +-----+
|      |      | 2.1. Đếm cạnh biên      |
+-----+      |      c_i = deg(u)      |
|  KẾT THÚC  |      |  C = Allreduce(SUM)  |
|  Xuất d(v)  |      |

```



7 Phân tích Độ phức tạp

Thành phần	Độ phức tạp trên mỗi Level
Tính toán Top-Down (tiên trình p_i)	$O\left(\frac{ F_l }{k} \cdot \overline{\deg}\right)$ — chia đều Frontier cho k tiến trình
Tính toán Bottom-Up (tiên trình p_i)	$O\left(\frac{n}{k}\right)$ — quét toàn bộ dải đỉnh, nhưng break sớm
Giao tiếp mạng (Allreduce)	$O(n \cdot \log k)$ — broadcast mảng kích thước n qua k máy

Nhận xét quan trọng: Chi phí giao tiếp $O(n \log k)$ **không phụ thuộc** vào việc đang duyệt Top-Down hay Bottom-Up. Nó xảy ra ở cuối **mỗi** level. Với đồ thị lớn (n lớn) trên mạng LAN chậm, chi phí này lấn át hoàn toàn thời gian tính toán — đây chính là nguyên nhân cốt lõi khiến Speedup tổng thể $S(P)$ sụp đổ trong thực nghiệm.

8 Tổng hợp Các Kỹ thuật Song song

#	Kỹ thuật	Tầng áp dụng	Mục đích
1	MPI (Distributed Memory)	Giữa 3 máy	Giao tiếp và đồng bộ dữ liệu qua mạng LAN
2	OpenMP (Shared Memory)	Bên trong mỗi máy	Tận dụng đa nhân CPU (4 hoặc 2 cores)
3	1D Block Partitioning	Chia tập đỉnh	Phân công đều đỉnh cho tiến trình
4	Graph Replication	Dữ liệu đồ thị	Loại bỏ giao tiếp Point-to-Point
5	Direction-Optimizing	Quyết định hướng	Cắt giảm số cạnh phải kiểm tra
6	Bitmap Frontier	Biểu diễn tập biên	Tra cứu $O(1)$, merge bằng OR
7	CAS Atomic	Tránh Race Condition	Đảm bảo chỉ 1 thread ghi vào $d(v)$
8	Dynamic Scheduling	Cân bằng tải OpenMP	Luồng xong sớm tự bốc việc tiếp
9	MPI_Allreduce (MAX, BOR, SUM)	Đồng bộ toàn cục	Hợp nhất kết quả từ 3 máy